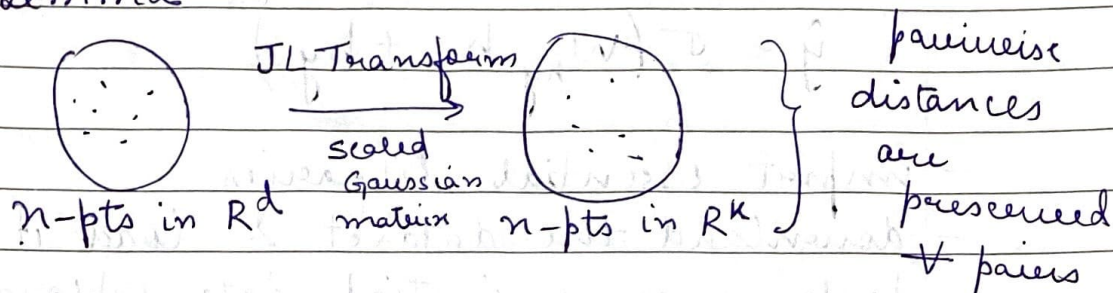


Week-3 ADS

L1 Approximate Nearest Neighbours

- JL Lemma



↓ applications $k \in O\left(\frac{\log n}{\epsilon^2}\right)$

nearest neighbour problem $\xrightarrow{\text{app.}}$ K-Means

⇓
closest images

⇓
map the user to movies
(recommender systems)

→ we use JL Lemma, $\therefore$ approximate nearest neighbours problem

$$\left. \begin{array}{ccc} x_1 & \dots & x_n & x_i \in R^d \\ \downarrow & & & \\ z_1 & \dots & z_n & z_i \in R^k \end{array} \right\} \begin{array}{l} \text{JL Transform} \\ \text{step} \Rightarrow \text{pre-processing} \\ \text{step for nearest neighbours} \end{array}$$

$$k \leq d$$

Preprocessing time (1-time) $\Rightarrow O(ndk)$
time

- Query processing time

$$q \in \mathbb{R}^d \xrightarrow[\substack{\text{JL Transform} \\ O(dk)}]{\quad} z_q \in \mathbb{R}^k \rightarrow \text{compute its distance in } k\text{-dim.}$$

1 distance $\Rightarrow O(k)$
 n distances $\Rightarrow O(nk)$

$O(dk + nk)$ time for 1 query pt.

In high dim.

$$q \in \mathbb{R}^d \Rightarrow O(nd) \gg O(dk + nk) \text{ because } k \ll d$$

- This works well only until your n is 'medium' sized $\sim 10^6$. If your n is much larger than d & k , then this becomes a problem for massively large datasets.
- Now, I ~~am~~ want somehow, that I should not depend on the $O(n)$ computations, while calculating the approximate neighbours problem.

L2 LSH (Local Sensitivity Hashing) - Intro.

- We have to find approx. ^{nearest} neighbours at scale (sub-linear in n).
 $\hookrightarrow$ we can't do JL & then linear search in n . $\Rightarrow$ that was the problem.
- We can build intuition from JL.
 $\hookrightarrow$ closer pts. ^(in d) get mapped to closer pts. _(in k)

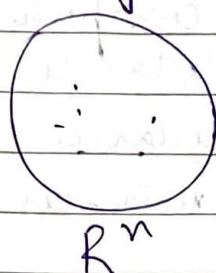
$a \Rightarrow$ pts. within a are similar

$c \cdot a \Rightarrow$ pts. beyond $(c \cdot a)$ are dissimilar

$p_1 \Rightarrow$ we want them to be hashed together / /

$p_2 \Rightarrow$ we don't want to hash them together.

- Hashing Concept



some mapping



Finite no. of buckets

when a new query comes, we have to check the buckets only. Check it & then check only those pts. of bucket will matter.

this is known as Locality Sensitivity

Hashing

- A family of hash func^{ns} H is cld (a, c, p_1, p_2)
 - sensitive for some $a > 0$, $c > 1$, $0 < p_2 < p_1 < 1$
- if for a randomly chosen funcⁿ $h \in H$, the following holds -
 - $\rightarrow$ for any 2 pts. $x, y \in \mathbb{R}^d$

$$\|x - y\| \leq a \Rightarrow \Pr_{h \in H} (h(x) = h(y)) \geq p_1$$

&

$$\|x - y\| \geq ca \Rightarrow \Pr_{h \in H} (h(x) = h(y)) \leq p_2$$

eg. $a=1$, $c=2$, $p_1=0.8$, $p_2=0.3$

closer guys

max-limit to be closer.

With the help of 'hash funcⁿ', I'll try to put closer guys in the same bucket.

L3: Hash Family

- LSH for Euclidean distance

$$H_w = \{h_{ab} \mid \forall a \in \mathbb{R}^d, b \in [0, w] \quad w > 0\}$$

Let $x \in \mathbb{R}^d$

define,
$$h_{ab}(x) = \left\lfloor \frac{a^T x + b}{w} \right\rfloor$$

$\mathbb{R}^d$ $\mathbb{R}^d$ $[0, w]$

$\rightarrow$ brings to an integer

$\rightarrow$ width param.

$\mathbb{R}^d \Rightarrow$ same no. on real line

done by $a^T x$.

the boxes are defined via w .

Randomness

random "direct" vector $a \sim N(0, I_d) \Rightarrow$ std. multivariate normal RV.

random "offset" $b \sim U[0, w]$

$w > 0 \Rightarrow$ bucket width $\Rightarrow$ this defines your family.

$\Downarrow$ controls how wide each hash bin is.

- The random vector a defines a line slicing through the space. 2 pts. x & y are hashed together if they both land in the same interval.

If x & y are close, their projections $a^T x$ & $a^T y$ are also close. They are likely to fall in the same interval, with same hash.

L4 LSH: Collision Analysis

- $x, y \in \mathbb{R}^d$ when $h(x) = h(y) \Rightarrow$ collision
- $P_{h \sim H} [h(x) = h(y)] = ?$
 $\hookrightarrow$ this is what we have to find out!
- Proof -

$$\underbrace{P_{h \sim H} [h(x) = h(y)]}_{\text{collision probab.}} = P_{\substack{a \sim N(0, I_d) \\ b \sim U[0, w]}} [h(x) = h(y)]$$

$$h(x) = \left\lfloor \frac{a^T x + b}{w} \right\rfloor, \quad h(y) = \left\lfloor \frac{a^T y + b}{w} \right\rfloor$$

first, fix $a \in \mathbb{R}^d$

$$\begin{aligned} P_{a,b} \left(\left\lfloor \frac{a^T x + b}{w} \right\rfloor = \left\lfloor \frac{a^T y + b}{w} \right\rfloor \right) & \quad \text{let } \begin{aligned} a^T x &= u, \\ a^T y &= v, \\ \Delta &= a^T (x - y) \end{aligned} \\ &= P_{a,b} \left(\left\lfloor \frac{u + b}{w} \right\rfloor = \left\lfloor \frac{v + b}{w} \right\rfloor \right) \quad v + \Delta = u \\ &= P_{a,b} \left(\left\lfloor \frac{v + \Delta + b}{w} \right\rfloor = \left\lfloor \frac{v + b}{w} \right\rfloor \right) \end{aligned}$$

so, necessarily, I want $|\Delta| < w$.

- $\rightarrow$ for collision to happen, $|\Delta| < w$, otherwise it jumps to adjacent blocks.
- $\rightarrow$ also, my b should lie btw. $[0, w - \Delta]$, so that we remain in the same box.

Now,

$$P_{a,b} (h(x) = h(y) | a) = \begin{cases} 1 - \frac{|\Delta|}{w} & \text{if } |\Delta| < w \\ 0 & \text{if } |\Delta| \geq w \end{cases}$$

if a is fixed,
 then probab. of collisions

$\swarrow \max(0, 1 - \frac{|\Delta|}{w})$
 $|\Delta| < w$

$$\Delta = a^T x - a^T y = a^T (x - y) \sim N(0, \|x - y\|^2)$$

$\downarrow$
 $N(0, I_d)$

$$P_u(h(x) = h(y)) = P_{\Delta \sim N(0, \|x - y\|^2)}$$

$$P_u(b \sim U(0, w) \mid (h(x) = h(y)))$$

$$= \int_{-\infty}^{\infty} \max\left(1, 1 - \frac{|t|}{w}\right) \frac{1}{\sqrt{2\pi} \|x - y\|} e^{-\frac{t^2}{2\|x - y\|^2}} dt$$

$\Delta \sim t$

$$\underbrace{P_u(h(x) = h(y) \mid t)}_{\text{collision probab.}} \cdot \underbrace{P_u(t)}_{\text{no closed form solution is available for this.}}$$

this is a monotonically decreasing funcⁿ of $\|x - y\|$.

$$= \frac{2}{\sqrt{2\pi} \|x - y\|} \int_0^{\infty} \max\left(0, 1 - \frac{t}{w}\right) e^{-\frac{t^2}{2\|x - y\|^2}} dt$$

as $\|x - y\| \downarrow$, collision probability increases

$$p_2 = f(cu) \quad p_1 = f(u) \quad \begin{matrix} u > 0 \\ c > 1 \end{matrix}$$

$u \uparrow$, probab. $\uparrow$

$$\therefore, 0 < p_2 < p_1 < 1$$

$\swarrow \searrow$
they should be fairly separable.

L5

L5H: TPR & FPR

TPR $\rightarrow$ True Positive Rate

FPR $\rightarrow$ False Positive Rate

- Recap

$H \rightarrow (\alpha, c, p_1, p_2)$ - sensitive

$$\begin{aligned} \alpha &> 0 \\ c &> 1 \\ 0 &\leq p_2 < p_1 \leq 1 \end{aligned}$$

$$\|x - y\| \leq \alpha \Rightarrow \Pr(h(x) = h(y)) \geq p_1 \Rightarrow \text{TPR}$$

$$\|x - y\| \geq c\alpha \Rightarrow \Pr(h(x) = h(y)) \leq p_2 \Rightarrow \text{FPR}$$

$$\text{TPR} > \text{FPR}$$

one eq. of H family

$$H_w = \left\{ h_{ab}(x) = \left\lfloor \frac{a^T x + b}{w} \right\rfloor \quad \forall a \in \mathbb{R}^d, b \in [0, w], w > 0 \right.$$

$\downarrow$ width

collision probab. $a \sim N(0, I_d), b \sim U[0, w]$

collision probab. distance b/w 2 pts. ($\|x - y\|$)

it is (α, c, p_1, p_2) - sensitive

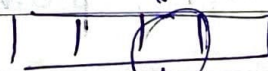
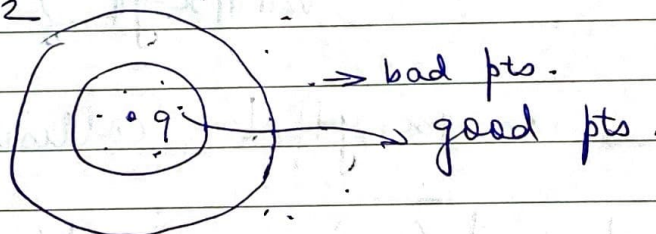
eg. $\alpha = 1, c = 2$

$$\begin{aligned} p_1 &= 0.8 \\ p_2 &= 0.4 \end{aligned}$$

$n = 10,000$ pts.

$T = 10 \Rightarrow$ no. of good pts. w.r.t query.

$q \rightarrow h(q)$



of bad pts. = 8000

how many pts. can I expect in this bucket?

→ each of the 10 pts. is going to collide with my q with probability 0.8.

$$\begin{aligned} \text{expected true + ves} &= 0.8 \times 10 \\ (\text{good pts.}) &= 8 \end{aligned}$$

$$\begin{aligned} \text{expected false + ves} &= p_2 \times \text{bad pts.} \\ (\text{bad pts.}) &= 0.4 \times 8000 \\ &= 3200 \end{aligned}$$

→ Though, actually I had to compute distances for only 10 pts., but I end up doing so for around 3208 pts. I am still in $O(n)$ computat^{ns}.

L6 LSH: Reducing FPR with 'And'

- we saw the problem above, let's tackle the FPR first. $\Rightarrow$ "AND" several hash func^{ns}.

$$h(x) = \left\lfloor \frac{a^T x + b}{w} \right\rfloor \quad \begin{aligned} a &\sim N(0, I_d) \\ b &\sim U[0, w] \end{aligned}$$

$$g(x) = [h_1(x), h_2(x), \dots, h_k(x)]$$

k -dim. space $\forall i, h_i(x) = \left\lfloor \frac{a_i^T x + b_i}{w} \right\rfloor$

$$q = [h_1(q), h_2(q), \dots, h_k(q)]$$

$$\text{Now, FPR} = \prod_{k=1}^k (p_2)^{k=2} \quad k=3$$

$$0.4 \quad 0.16 \quad 0.064 \quad \dots$$

what should be the value of k ?

we reduce drastically.

- For any k ,

$$\text{expected no. of FP collisions} \approx n \cdot (p_2)^k$$

I don't want it to depend on n . But I can choose k , right?

choose k s.t.

$$n \cdot (p_2)^k = 1 \Rightarrow (p_2)^k = \frac{1}{n}$$
$$k \log p_2 = \log(1/n)$$

$$k = \frac{\log(1/n)}{\log(p_2)} = -\frac{\log(n)}{-\log(1/p_2)} = \frac{\log(n)}{\log(1/p_2)}$$

$$\boxed{k = \frac{\log(n)}{\log(1/p_2)}}$$

for ex. eg.

$$n = 10000$$

$$p_2 = 0.4$$

$$k \approx 10$$

$$\text{FPR} = 10000 \times (0.4)^{10} = 0.00010486 \times 10000$$
$$\approx 1.04$$

- In this way, TPR also suffers.

$$(0.8)^{10} = 0.1074 \times 10 \approx 1.074$$

- Distance computation is now very good. But, now we have increased our chance of actually finding the true nearest neighbour.
→ This is a worse situation than before.

L7

Increasing TPR with "OR"

$$g_1(x) = [h_{11}(x) \ h_{12}(x) \ \dots \ h_{1k}(x)] \quad T_1$$

$$g_2(x) = [h_{21}(x) \ h_{22}(x) \ \dots \ h_{2k}(x)] \quad T_2$$

$$g_l(x) = [h_{l1}(x) \ h_{l2}(x) \ \dots \ h_{lk}(x)] \quad T_l$$

x & y collide if at least 1 table matches.

- because of AND $\rightarrow$ FPR & TPR reduces.
- TPR

$\rightarrow$ chance that a q gets hashed to the same bucket as a 'good' pt.

$= 1 - (\text{chance the } q \text{ gets not hashed to a good pt.})$

$= 1 - \text{none of the } l \text{ tables matched.}$

$$= 1 - (1 - p_i^k)^L$$

at least one of

$$P(g_1(q) = g_1(x), g_2(q) = g_2(x) \dots g_k(q) = g_k(x))$$

$$= 1 - P \left(\begin{array}{l} g_1(q) \neq g_1(x) \text{ and} \\ g_2(q) \neq g_2(x) \text{ and} \dots \\ g_L(q) \neq g_L(x) \end{array} \right)$$

$$= 1 - \prod_{i=1}^L (1 - P(g_i(q) = g_i(x)))$$

$$TPR = 1 - (1 - p_1^k)^L$$

by same argument,

$$FPR = 1 - (1 - p_2^k)^L$$

$$- \text{TPR} = (1 - (1 - p_1^k)^L)$$

we know, $k = \frac{\log(n)}{\log(1/p_2)}$

do some calculations

$$\boxed{\rho = \frac{\log(1/p_1)}{\log(1/p_2)}}$$

$$p_1^k = \exp[\log n^{-\rho}]$$

$$p_1^k = n^{-\rho} \Rightarrow \text{TPR} = (1 - (1 - \underbrace{n^{-\rho}}_{p_1^k})^L)$$

now, we have to see the value of L . $k \rightarrow$ our choice

$$\boxed{L = O(n^\rho)}$$

$$\Rightarrow L = O n^\rho$$

$\rightarrow$ some constant

$$\text{TPR} = (1 - (1 - n^{-\rho})^{O n^\rho})$$

for large n , $\approx 1 - \left(\frac{1}{e}\right)^\theta$

$$\boxed{\text{TPR} \approx 1 - e^{-\theta}}$$

$$\theta = 1, 0.63$$

$$\theta = 2, 0.86$$

$$\theta = 3, 0.95$$

- FPR, after OR operation -

$$\begin{aligned} \text{FPR} &= (1 - (1 - p_2^k)^L) \\ &= (1 - (1 - \frac{1}{n})^L) \end{aligned}$$

$$L = O n^\rho$$

$$\rho = \frac{\log(1/p_1)}{\log(1/p_2)}$$

for large n ,

$$FPR = 1 - \left(1 - \frac{1}{n}\right)^L \approx 1 - e^{-L/n}$$

$$FPR = 1 - e^{-O(n^{p-1})}$$

$$\boxed{FPR \approx O(n^{p-1})} \xrightarrow{\substack{n^{p-1} \rightarrow 0 \\ \text{by first order} \\ \text{Taylor approximation}}}$$

$$\text{no. of bad guys} = O(n^p)$$

L8

LSH: Summary & Analysis

$$k = \frac{\log(n)}{\log(1/p_2)}$$

$$L = O(n^p)$$

$$p = \frac{\log(1/p_1)}{\log(1/p_2)}$$

$$\Rightarrow \boxed{\begin{array}{l} TPR = 1 - e^{-O} \\ FPR = O(n^p) \end{array}}$$

- Our old q. $n = 10K$

$T = 10$ (good), $B = 8000$ (bad)

if $\alpha = 2$

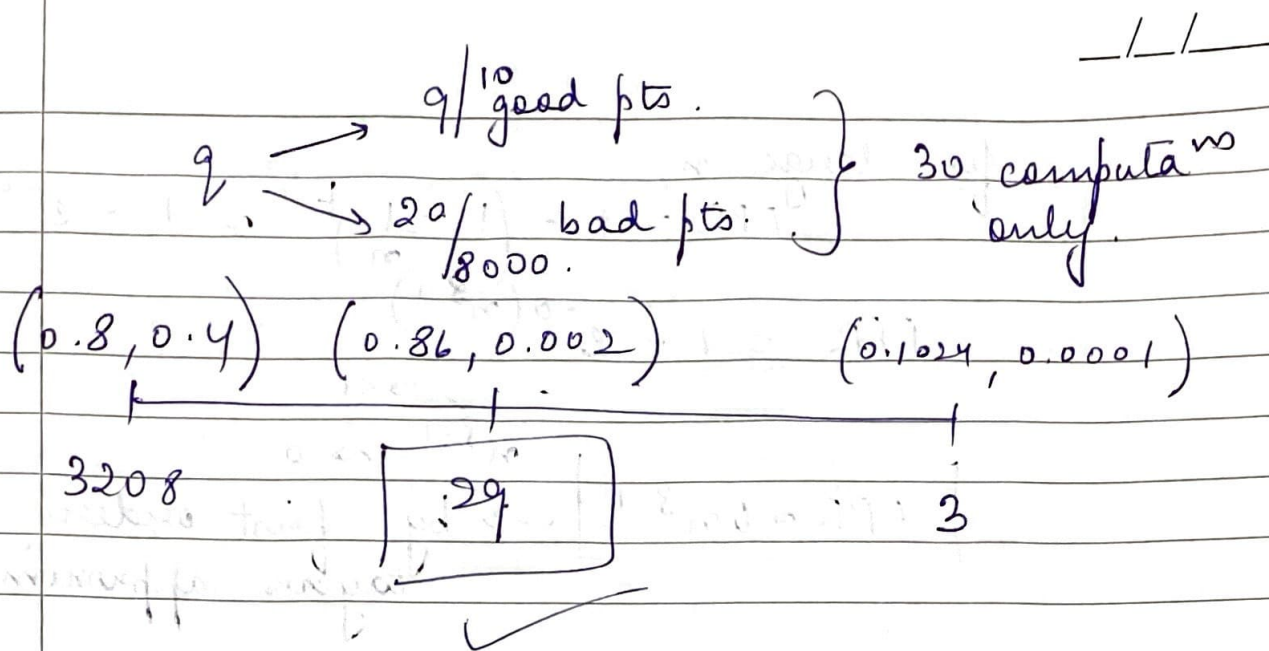
$$TPR = 0.86$$

$$\# \text{ good} = 8.6 \approx 9$$

$$FPR = 2 \cdot n^p$$

$$\# \text{ bad} = (8000)^{0.24345} \times 2 = 20$$

$$p = \frac{\log(1/0.8)}{\log(1/0.4)} = 0.243$$



→ we critically depend on $\beta \rightarrow$ controls TPR & query complexity as well.

⌋
(depends on no. of tables L)

$n^\beta \leftarrow$ sub-linearity.

it summarizes the entire hash funcⁿ by depending on p_1 & p_2 .

We want to have β as low as possible.

- we have to design hash funcⁿ such that we get good separaⁿ btw. p_1 & p_2 .
- In Datac style hash funcⁿ, p_1 & p_2 depends on $w \rightarrow$ inpt. parameter.